

Sommario esecutivo

Creiamo un prototipo basato su *Atlas Reactor*, un FPS strategico 4v4 a turni simultanei, per imparare Unreal Engine. Ogni turno prevede una **modalità decisione** (20s per scegliere mosse) seguita da una **modalità di risoluzione** suddivisa in quattro fasi (Prep, Dash, Blast, Move) [44†L139-L148] [1†L48-L57]. Durante ciascuna fase, le azioni di tutti i giocatori vengono calcolate simultaneamente ma applicate in ordine prefissato [44†L149-L155] [1†L48-L57]. Il progetto includerà sistemi chiave come **rete multiplayer** (replicazione client-server [9†L56-L64]), simulazione deterministica (indispensabile per turni simultanei), rollback e predizione degli input per la latenza, matchmaking e lobby (es. EOS o Photon), interfaccia utente (UMG), animazioni, VFX e audio. L'**art pipeline** prevede asset 3D (modelli, materiali, texture, rigging, animazioni) importati via FBX [28†L164-L172] [28†L194-L202]. Il *level design* userà una visuale top-down/isometrica (un template Unreal dedicato è già disponibile [26†L99-L108]). I personaggi avranno ruoli distinti (es. tanker, supporto, DPS) con abilità uniche e bilanciate da cooldown, e un sistema di progressione o monetizzazione (es. sblocco cosmetici). Come stack: Unreal Engine 5.8 (ultima versione) [13†L0-L3], con Blueprints e C++ (o plugin C# come UnrealCLR/MonoUE o, in alternativa, Unity). Confronteremo opzioni C++ vs C#, *framework di rete* (UE nativa, EOS, Photon), *architetture multiplayer* (client-server vs peer-to-peer vs lockstep) in tabelle sintetiche. Infine, definiamo milestone di apprendimento (p.es. setup, logica turni, rete, UI, AI) con tempi stimati, risorse didattiche (documentazione UE, tutorial, guide community), diagrammi (flowchart del loop di gioco e diagramma di Gantt) e snippet di codice/pseudocodice essenziali. Variabili aperte includono budget, team, piattaforme target e stile artistico.

Loop di gioco e meccaniche chiave

In *Atlas Reactor*, ogni turno è simultaneo: i 4 giocatori per team **preparano le azioni** in 20 secondi (Decision Mode) e poi il gioco **resolve** le azioni in sequenza di fasi (Resolution Mode) [44†L139-L148] [1†L48-L57]. Le fasi tipiche sono: **Prep** (trappole, buff/debuff), **Dash** (schivate/posizionamenti veloci), **Blast** (attacchi primari) e **Move** (spostamenti finali) [1†L48-L57] [4†L59-L63]. Ogni abilità ha un "costo" di fase o azione libera; quasi tutte le abilità hanno cooldown per bilanciare il potere [1†L99-L107] [4†L59-L63]. Vince il team che elimina 5 avversari per primo o, dopo 20 turni, ha più kill [1†L59-L62]. Nonostante la complessità, il gioco risulta tattico: ogni giocatore deve prevedere le mosse avversarie (es. un nemico potrebbe *Dashare* fuori dal tiro) e reagire di conseguenza [1†L66-L74] [4†L59-L63]. In pratica, come recita Wikipedia, «all'interno di ogni fase tutte le azioni dei giocatori sono calcolate simultaneamente ma mostrate sequenzialmente» [44†L149-L155], per cui un personaggio colpito può apparire ancora in piedi fino al suo turno di azione completato. Questo modello richiede un **simulatore deterministico** (per garantire che data la stessa sequenza di input il risultato sia identico su tutti i client) o un server autorevole che esegue un lockstep centralizzato. In entrambi i casi, durante la fase di input il server (o host) raccoglie le azioni di ogni client (scegliendo abilità e direzione) e poi le "gioca" nell'ordine predeterminato.

flowchart TB

```
Start[[Inizio Turno]] --> DecisionMode{{Modalità Decisione (20s)}}
DecisionMode --> Prep[Fase Prep]
Prep --> Dash[Fase Dash]
Dash --> Blast[Fase Blast]
```

```
Blast --> Move[Fase Move]
Move --> CheckWin{Obiettivo raggiunto?}
CheckWin -- No --> NextTurn[Turno Successivo]
CheckWin -- Sì --> EndGame[Fine Partita]
```

Sistemi di gioco necessari

- **Rete multiplayer** (architettura client-server): Unreal utilizza un modello **client-server** autorevole [9†L56-L64] [9†L72-L80] . I client inviano comandi (RPC) al server, che elabora la simulazione e replica gli aggiornamenti di stato (posizioni, salute, ecc.) a tutti i client [9†L56-L64] [9†L72-L80] . Le classi *Character/Pawn* UE supportano la replicazione nativa: il componente di movimento replica le posizioni e la fisica (se abilitata) [8†L60-L68] . Variabili di gameplay rilevanti (es. vita) si sincronizzano mediante *UPROPERTY(Replicated)* o *RepNotify* [8†L60-L68] . L'interfaccia utente (UMG) non è replicata in sé, ma aggiorna la UI locale sulla base delle variabili di gioco replicate (es. salute, munizioni).
- **Simulazione deterministica**: fondamentale perché ogni client (o il server) calcoli gli stessi esiti. Unreal non garantisce determinismo sui motori fisici integrati (Chaos o PhysX) [12†L91-L99] . Quindi occorre limitare la fisica non deterministica (es. usare spostamenti cinematici o rigging proprio). L'alternativa è far simulare **solo al server**, replicando semplicemente la posizione dei personaggi: i client possono prevedere il proprio movimento localmente (client-side prediction) e correggere quando arriva il dato dal server.
- **Rollback e predizione**: per ridurre la latenza di risposta, si può implementare una logica di *client-side prediction* (il client simula in anticipo le proprie azioni) e *rollback* (ripristino di uno stato precedente se arrivano comandi tardivi) [15†L23-L30] [12†L91-L99] . Tuttavia, il rollback richiede tick deterministici custom e solitamente è complesso in UE; le fonti sconsigliano l'uso di Blueprint per questo (meglio C++) [15†L23-L30] . Alcuni suggeriscono che il sistema **Gameplay Ability System (GAS)** di Unreal gestisca meccanismi simili internamente, ma senza documentazione approfondita. In pratica, si potrebbe cominciare senza rollback e introdurlo solo in caso di problemi di sincronizzazione gravi.
- **Matchmaking e lobby**: per la fase di ricerca partita e organizzazione del match. Si possono usare **Epic Online Services (EOS)** o **Photon**. EOS è integrato con UE via plugin (Online Subsystem EOS) e offre gratuitamente funzionalità cross-platform, account Epic, lobby e matchmaking [46†L13-L22] [21†L12-L20] . Photon offre SDK multiplatforma che si integra con UE (solo C++, come indicato) [23†L194-L202] [23†L226-L234] . EOS è ideale per cross-play e integrazione con il mondo Unreal (gratuito, backend scalabile [21†L12-L20]), mentre Photon può essere usato per sessioni istantanee (richiede configurazione SDK e hosting esterno, gratuito fino a certi limiti). La scelta dipende da esigenze (es. EOS ha statitica free e supporto Fortnite/Unreal, Photon è maturo e indipendente).
- **Interfaccia Utente (UI)**: con UMG si realizzeranno menu, HUD di gioco, selettore abilità, timer turni, scoreboard, ecc. La UI opera localmente su ogni client e si sincronizza alle variabili di gioco replicate (es. tempo rimasto, salute, uccisioni). Unreal UMG è documentato e versatile per gli elementi necessari.
- **Animazione**: i personaggi avranno scheletri e animazioni, importate (FBX/Collada) da strumenti come Blender/Maya. In rete, le animazioni si basano sulle variabili replicate (es. velocità del *CharacterMovement*). Unreal consiglia di non replicare direttamente l'animazione ma usare *AnimBlueprint* che legge variabili (stato movimento, salute) aggiornate dalla rete [8†L60-L68] . Effetti di montaggio (*AnimMontage*) possono essere attivati via server e client.
- **VFX e audio**: effetti particellari (Niagara/Blueprint) e suoni (Audio Engine UE) si attivano localmente quando cambiano eventi di gioco replicati. Ad esempio, un'abilità che infligge danno invia evento al server, il server riduce vita e invia RPC per far giocare l'animazione d'impatto; tutti

i client riproducono allora VFX/FX collegati all'evento. Non è necessario replicare dati dei VFX, ma solo trigger di gioco.

- **Matchmaking e lobbies:** usando EOS o Photon (o Steamworks) per far accoppiare i giocatori. Unreal Online Subsystem (OSS) e plugin EOS semplificano il login e la formazione di lobby [46†L13-L22] . Photon supporta “stanze” già pronte e hosting decentralizzato. La scelta influenza la complessità dell'integrazione: OSS EOS è ben documentato su devdocs, Photon richiede l'SDK C++ dedicato [23†L194-L202] .

Pipeline artistica e tipi di asset

Per gli asset 3D si possono utilizzare tool industry-standard: modellazione con Blender/Maya/3dsMax, texturing con Substance Painter/Designer, rigging e animazione nello stesso strumento o in MotionBuilder. I file 3D vengono esportati in **FBX** (UE consiglia versione 2018) e importati con il Content Browser [28†L164-L172] . UE crea *Static Mesh* o *Skeletal Mesh* (.uasset) corrispondenti. Le texture (PNG, TGA) si importano similmente [28†L125-L134] . Successivamente si creano *Materiali* in Unreal usando l'Editor Material (es. shader per metallo, vetro, luci). Particelle e VFX si possono realizzare con **Niagara** o cascades (ambienti, esplosioni). Audio (effetti sonori e musiche) si importano come WAV/OGG e si gestiscono tramite l'Audio Engine. L'**Artist Quick Start** di Unreal mostra come organizzare le cartelle (es. *Textures*, *Meshes*, *Animations*) e importare asset [28†L90-L99] [28†L164-L172] . Ad esempio, la pipeline include: 1) modellare static mesh; 2) unwrap UV; 3) creare materiale/diffuse/specular; 4) esportare FBX; 5) importare in Unreal (spuntare “Import Materials/Textures”) [28†L164-L172]

[28†L194-L202] . Asset complessi (personaggi) richiedono rigging e animazioni multiple, poi importate come Skeletal Mesh con state machine in AnimBlueprint. Per accelerare, si possono usare asset store (es. Marketplace UE o Quixel Bridge fornisce modelli e texture di qualità).

Level design e controllo telecamera

Il gioco si svolge in arene chiuse, viste dall'alto o in prospettiva isometrica. Unreal ha un *Top Down Template* che fornisce un sistema base di visuale dall'alto [26†L99-L108] . In particolare, il *Variant Strategy* usa una telecamera **ortografica isometrica** con zoom e pan regolabili [26†L99-L108] . Possiamo partire da questo template o replicarne la logica: la telecamera si posiziona sopra il campo (angolo ~45° o direttamente dall'alto) e segue i movimenti del puntatore o del personaggio selezionato. I controlli di movimento tipici per un gioco come Atlas Reactor sono a “click point”: il giocatore clicca dove muoversi o usare abilità (mappa tratteggiata/grid). In alternativa, si può usare WASD per spostare la camera (come in RTS) e clic sinistro per selezionare unità e destro per destinazione (usato in Strategy variant [26†L99-L108]). Il *TopDown Blueprint* di Unreal mostra come aggiornare la posizione della camera e gestire ingrandimenti [26†L99-L108] . Il level design delle mappe dovrà tenere conto di bilanciamento e dinamiche di gioco: arene simmetriche, ostacoli copribili, altezze o porte (come in Atlas Reactor), punti di interesse (buff o coperture). È utile disporre i percorsi in griglia (anche invisibile) per facilitare il movimento a turni. In fase iniziale, usare un asset grafico semplice (blockout) e poi abbellirlo con ambientazione tematica. In sintesi, *camera iso*, controlli point-and-click/mouse+UI, e mappe progettate per tattiche di squadra sono raccomandati.

Design personaggi, abilità e progressione

Si definiscono alcune classi archetipiche (tank, supporto, dps, cecchino, ecc.) con abilità uniche. Ogni personaggio (freelancer) ha 3-4 abilità: p.es. un colpo principale (Blast), un dash (schivata) e qualche skill prep (scudo, trappola o cura). Abilità potenti hanno cooldown alti [1†L100-L109] . Importante il sistema di *catalysts* di Atlas Reactor: bonus che modificano le abilità (es. +danno, +velocità di ricarica). Si può riprendere un concetto simile con perk equipaggiabili. Il bilanciamento richiede iterazioni:

assumiamo valori base (HP, danno, cooldown) e aggiustiamoli durante playtest. La progressione può essere progettata come unlock cosmetici o ricompense sezionali (Atlas Reactor aveva stagioni con missioni 【44†L161-L170】). Monetizzazione opzionale: skin dei personaggi, emote, o DLC di mappe; evitare pay-to-win (solo estetico o pass stagionale). Nel prototipo MVP si può saltare la monetizzazione e focalizzarsi sulle meccaniche fondamentali.

Stack tecnologico consigliato

Opzione	Unreal C++	Unreal C# (CLR plugin)	Unity (C#)
Supporto UE	<i>Nativo</i> , massima integrazione (repliche, editor) 【8†L60-L68】	Plugin di terze parti (UnrealCLR, MonoUE) meno maturo; limita l'uso di Blueprints	Integrazione completa con C#, ma resti fuori dall'ecosistema UE
Learning curve	Più ripido per chi non conosce C++; massima potenza e controllo	Più facile se già si conosce C#; può mancare documentazione ufficiale	Simile a C#, con Unity in lingua inglese o risorse italiane
Prestazioni	Maggiori performance, accesso completo ai codici sorgente UE	Overhead del plugin; incertezza su manutenzione futura	Buone, ma engine diverso (target tecnologie diverse)
Blueprint	Integra Blueprints per design rapido	Possibile uso di Blueprints normalmente; C# al posto di C++	Usa il proprio sistema (C# + Prefab)
Plugin/Replicazione	Tutte le funzionalità native UE disponibili (AI, rete, Chaos)	Bisogna verificare compatibilità plugin di rete (OSS EOS)	Integrazione con Photon/Firebase/Netcode diversa; pipeline asset differente

Per il prototipo si **consiglia Unreal Engine 5.8** (ultimo rilascio, già disponibile 【13†L0-L3】) perché ha le novità grafiche e di rete più recenti. Si può lavorare ibridamente: programmare in C++ le parti core (logica di turni, netcode) e usare **Blueprints** per prototipare in fretta UI o comportamenti. Un plugin C# esiste (UnrealCLR), ma è meno sicuro a livello di supporto e debugging. In alternativa, se si vuole restare in C#, valutare Unity con Photon o MLAPI: Unity è più orientato C#, ma per imparare Unreal conviene usare UE.

Framework di rete: Unreal Networking built-in (replica/RPC) è la scelta naturale 【9†L56-L64】 【8†L60-L68】 . Per matchmaking e servizi cross-platform: **EOS** (gratuito, multi-piattaforma, account Epic) 【21†L12-L20】 【46†L13-L22】 , oppure **Photon** (SDK real-time, open-source limitato, free plan) 【23†L194-L202】 【23†L226-L234】 . EOS offre l'oss fallback integrato e UI per sessioni/lobby, Photon è plug-in C++ standalone (serve un server o PUN). **Frostbite** etc. non rilevanti.

Fisica: per unità piccole (personaggi) usare carattere cinematografico (no cloth/rigidbody complessi). Il movimento basterà via *CharacterMovementComponent* (UE). Evitare fisica complessa perché non deterministica 【12†L91-L99】 .

AI: i bot possono usare il *NavMesh* e *Behavior Tree* UE per prendere decisioni di base. Inizialmente, abilità e movimenti dei bot possono essere semplici (es. "sparo quando in vista"): utile per playtest.

Strumenti e workflow: usare Visual Studio o Rider per codice C++. Versionamento (Git) è consigliato. Il motore UE5 include editor grafico, debugger Blueprints, Profiling, etc. Consultare la documentazione ufficiale UE (in inglese) e guide community.

Pianificazione e milestone didattiche

Definiamo tappe incrementali, ciascuna con obiettivi chiari e risorse di apprendimento. I tempi stimati sono indicativi (dipendono dall'esperienza individuale). Ad esempio:

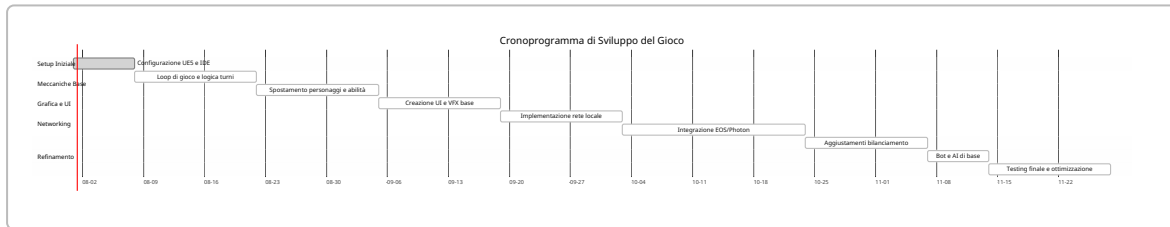
- **Setup ambiente e progetto (1 settimana):** installare Unreal Engine 5.8, Visual Studio; creare progetto base (template *Top Down C++* con Starter Content). Tutorial utile: [Quick Start Multiplayer \(UE5\)](#) [8†L11-L18] .
- **Logica turni base (2 settimane):** implementare il loop del turno in locale. Creare classi per *FaseTurno* e *AzioneGiocatore*; usare timer di 20s per input. Non ancora rete: simulazione local/second player. Esempi: [Blueprint API TurnBased](#) (sezione teorica).
- **Movimento e abilità (2 settimane):** aggiungere spostamenti dei personaggi (navmesh o griglia) e primitive abilità (es. proiettile, dash). Usare *CharacterMovement* per la fisica base. Tutorial: *Top Down Template* di Unreal spiega il movimento e selezione mult-unità [26†L99-L108] .
- **UI e VFX (1-2 settimane):** creare interfaccia (barra abilità, timer, conto-kill) con UMG. Implementare semplici VFX di impatto (Niagara) e suoni. Risorse: [Unreal UMG Tutorials su Unreal Dev Blog](#).
- **Networking locale (2 settimane):** abilitare sessione multiplayer in locale (due client sullo stesso PC) per provare la replica. Seguire *Multiplayer Quick Start* [8†L11-L18] [8†L60-L68] . Replicare movimenti e variabili (usa `UFUNCTION(Server)` e `ReplicatedUsing`).
- **Networking online (3 settimane):** scegliere framework (es. EOS o Photon). Configurare plugin EOS in UE [46†L13-L22] e testare partite su LAN. Implementare la raccolta sincrona delle azioni: ad ogni inizio turno i client inviano al server i propri input, il server li archivia e attende gli altri, poi chiama la funzione `SimulaTurno()` . Pseudocodice base:

```
// Riceve input da ogni client
UFUNCTION(Server, Reliable)
void ServerSubmitActions(const TArray<ActionData>& Inputs);

// Simula il turno sul server
void SimulaTurno() {
    per ogni fase di turno (Prep, Dash, Blast, Move) {
        per ogni azione di quella fase:
            ApplicareEffetto(azione);
    }
    // Invia aggiornamenti client
    MulticastAggiornaStato(...);
}
```

- **Bilanciamento e AI (2 settimane):** integrare comportamenti bot basici per completare la partita in mancanza di giocatori umani. Playtest e calibrare statistiche (velocità, HP, danni).
- **Ottimizzazione e testing (2 settimane):** correggere bug di rete (es. lag), ottimizzare performance (ridurre overdraw, es. con *MIP map* delle texture).

Ogni milestone può includere tutorial/documentazione (in inglese e italiano ove possibile). In italiano: il sito della documentazione UE ha alcune traduzioni (p.es. guida *Top Down* in parte tradotta [\[26†L99-L108\]](#)), canali YouTube italiani (es. *GameDev Italia*, *Matteo Salerno*) e il libro “Guida a Unreal Engine” (non ufficiale). In inglese: la documentazione Epic, blog e forum (es. risposta su rollback [\[15†L23-L30\]](#)).



Confronti e considerazioni

C++ vs C# (Unreal): C++ è *nativo* in UE, con documentazione completa e performance massime [\[8†L60-L68\]](#) . Richiede tempo per imparare la sintassi UE (UCLASS, UPROPERTY). C# in UE è possibile via plugin (UnrealCLR) ma è community-driven: meno stabilità e integrazione. Con Unity (C#) si rinuncia alla base Unreal, pur facilitando lo sviluppo se si preferisce C#. La tabella sopra riassume i pro/contro.

Framework di rete:

| Framework | Integrazione UE | Linguaggio | Note principali |

|-----|-----|-----|-----| | **UE Networking** | Nativo UE | C++/Blueprint | Client-server integrato, facilità di replica e RPC [\[9†L56-L64\]](#) [\[8†L60-L68\]](#) | | **Epic Online Services (EOS)** | Plugin UE (OSS EOS) | C++/Blueprint | Servizi gratuiti (matchmaking, lobbies, cross-play) [\[21†L12-L20\]](#) [\[46†L13-L22\]](#) | | **Photon Realtime** | SDK C++ esterno | C++ | Soluzione cross-engine, free limitato; server P2P possibile [\[23†L194-L202\]](#) [\[23†L226-L234\]](#) | | **Custom Lockstep** | (No UE nativo) | Qualsiasi (deterministico) | Utile in RTS, complesso in UE; richiede simulazione identica per tutti. |

Il più semplice per iniziare è UE Networking integrato (client-server). EOS/Photon entrano in gioco per servizi aggiuntivi o crossplay. **Architetture multiplayer:**

- *Client-Server (server autoritario):* il server calcola il gioco e invia stati. Veloce per pochi giocatori (8 in questo caso). Consente controllo anti-cheat. Unreal è basato su questo [\[9†L56-L64\]](#) .
- *Peer-to-Peer (P2P):* tutti pari, ognuno invia direttamente ad altri. Usato storicamente in RTS lockstep. Problema: latenza dipende dal peggior collegamento e complesse NATting. Non consigliato per un progetto UE moderno.
- *Lockstep deterministico:* ogni client simula il gioco identicamente a partire dagli stessi input; usato in RTS classici (Age of Empires). Richiede determinismo rigido ed aspettare i comandi di tutti i client ogni tick [\[11†L88-L96\]](#) [\[12†L91-L99\]](#) . In UE impraticabile per via della fisica non deterministica [\[12†L91-L99\]](#) .

Il **MVP consigliato**: supportare due giocatori (locale o in rete) e un carrello di 2-3 abilità, mappa semplice, UI basica (seleziona abilità, timer, vita). Questo copre il core loop. Le funzionalità avanzate (AI, matchmaking esteso, rollback avanzato) possono arrivare in step successivi.

Rischi e trade-off: la sfida maggiore è il network. Possiamo iniziare con simulazione turn-based locale (senza rete) per validare meccaniche, poi integrare gradualmente la rete. La mancanza di determinismo in UE impone di fidarsi del server o accettare correzioni visive (rubber-banding) [\[12†L91-L99\]](#) . Fare troppe feature contemporaneamente (grafica complessa, rollback, multiplayer competitivo) aumenta i

rischi. Meglio iterare partendo da un prototipo giocabile. Nel report abbiamo ipotizzato **requisiti generali non noti** (budget sconosciuto, team singolo, target principalmente PC, stile artistico generico): questi rimangono variabili aperte da definire con il progetto.

Risorse consigliate: la documentazione ufficiale UE è molto dettagliata e va privilegiata (es. rete [【9†L56-L64】](#) , top-down template [【26†L99-L108】](#) , online subsystem [【46†L13-L22】](#)). In italiano, si trovano guide comunitarie e tutorial video (ad es. canali YouTube italiani su UE5, blog come *TutsPlus* con traduzioni). Ad esempio, il Top Down Template ha una pagina UE in italiano (vedi la parte *Strategy Variant* per la telecamera ortografica [【26†L99-L108】](#)). Per la fisica deterministica, leggere forum UE [【12†L91-L99】](#) ; per il networking e input, i tutorial multiplayer UE di Epic. La tabella seguente riassume alcune risorse:

Argomento	Risorse	Lingua
Rete UE (replica)	<i>Networking Quick Start</i> (Epic UE docs) 【8†L11-L18】	Inglese
Simulazione Turni	Wiki <i>Atlas Reactor</i> (meccaniche) 【44†L139-L148】 ; blog analisi 【1†L48-L57】	Italiano/ INGLESE
Top-Down / Telecamera	UE Doc <i>Top Down Template</i> 【26†L99-L108】	Italiano (parziale)
Animazioni / VFX	UE Docs su <i>AnimBlueprint</i> e <i>Niagara</i>	Inglese
EOS/Photon	UE Doc <i>OSS EOS Plugin</i> 【46†L13-L22】 ; Photon UE Docs 【23†L194-L202】	Inglese
Tutorial UE Italia	YouTube: <i>GameDev Italia</i> , <i>Matteo Salerno</i>	Italiano

Ogni milestone (setup, logica, rete, UI, AI, test) deve avere obiettivi concreti e codice di esempio. Ad esempio, per la **rappresentazione turni** si possono definire strutture dati (es. struct `Action{ int playerId; AbilityType; FVector target; }`) e funzioni RPC. Per la **raccolta input** si potrebbero usare array sincronizzati di `Action` sul server e poi processarli a turno. Il snippet sotto illustra schematicamente la risoluzione di un turno:

```
void AGameModeVR::SimulaTurno() {
    // Esempio pseudocodice su server
    for (Phase phase : {Prep, Dash, Blast, Move}) {
        for (Action act : Azioni[phase]) {
            ApplyActionEffects(act); // Calcola danni, spostamenti, buff
        }
    }
    CheckEndGame(); // Verifica vittoria/sconfitta
    RpcAggiornaStatoAClient(); // Invia a tutti lo stato finale del turno
}
```

In conclusione, seguendo questo piano si gettano le basi sia per un'esperienza di gioco solida (meccaniche e sistemi core) sia per un percorso didattico graduale. L'approccio gen Z aiuta a mantenere lo stile leggero ma operativo: si procede per prove e iterazioni, facendo leva sulle potenti feature di Unreal e consultando le fonti ufficiali [【9†L56-L64】](#) [【26†L99-L108】](#) . Se si incontrano lacune, si ricorre a forum e tutorial dedicati (idealmente italiani) per chiarimenti.

Fonti: documentazione Unreal Engine 【8†L11-L18】 【9†L56-L64】 【26†L99-L108】 【46†L13-L22】 ,
analisi di *Atlas Reactor* 【1†L48-L57】 【44†L139-L148】 , discussioni tecniche Unreal (replicazione,
rollback) 【8†L60-L68】 【12†L91-L99】 【15†L23-L30】 .
